Stash — Technical Brief

A native macOS menu bar app for saving project references. This document describes everything that exists today — product behaviour, architecture, data, every source file, the tricky parts, and where it can be made more efficient — so an AI assistant (or a developer) can understand the whole program before changing it. Snapshot: October 2026 · ~9,300 lines of Swift · 38 source files.

> **How to use this brief (for the AI reading it).** Treat it as the ground truth of the current design. When you propose a change: (1) name the files you would touch, (2) keep the user-facing behaviours listed in section 4 unless told otherwise, (3) respect the constraints in section 9 (non-activating panels, iCloud-hidden library folder, mirror-as-truth for files), and (4) prefer changes that reduce work on the main thread. The owner is a designer, not a programmer: explain trade-offs in plain words, and give step-by-step instructions.

---

1. What Stash is

Stash is a fast "scrapbook" for creative projects that lives in the macOS menu bar. You throw links, images, notes, files, videos and audio into it (paste, drag, a global shortcut, or right after a screenshot) and it keeps them organised by **project** (browser-style tabs) and by **folders** (nested, with typed folders that sort things automatically). Everything is also kept as **real files** in a hidden folder, `~/Documents/.Stash`, so nothing is locked inside the app.

- **Two windows.** A compact pop-up (330×440) that hangs from the menu bar icon, and a big movable window (⤢). ⌥Space toggles. Both are non-activating panels, so the app you were in stays in front.
- **Projects** = tabs. "Not Selected" holds things in no project. Only pinned + recently opened projects get tabs; a ▾ list shows all.
- **Folders** can nest. Styles: Folder (classic), Kraft, Pastel, Midnight, and typed folders **Linker** (links), **PhotoBook** (images), **Notepad** (notes), **Files** (other files). The wand "organizes" loose things into the typed folders.
- **Built-in media tools.** Convert video/audio to MP3/MP4 (AVFoundation + a bundled LAME encoder) and download YouTube etc. as MP3/MP4 (bundled `yt-dlp`), with no external installs.
- **Finder-like behaviour** throughout: selection, rename, colour tags, Get Info (⌘I), Quick Look (Space), drag anything in or out, ⌘Z undo.

---

2. Tech stack and build

Item	Detail
Language / UI	Swift 5.10, SwiftUI for views, AppKit for windows, events, menus, drag sessions, Quick Look, pasteboard.
Package	Swift Package Manager executable target `Stash` (`Package.swift`, platform macOS 14+). No third-party Swift packages.
Bundled tools	`Resources/tools/lame` (0.3 MB, built from LAME 3.100 source), `Resources/tools/yt-dlp` (35 MB standalone binary). yt-dlp is copied to `~/Library/Application Support/Stash/tools` so it can self-update.

Build	`./build.sh` → `swift build -c release`, assembles `Stash.app` in a temp folder (outside iCloud Desktop, whose file attributes break codesign), clears xattrs, ad-hoc signs, copies to `build/Stash.app`. `./build.sh install` also replaces `/Applications/Stash.app` and relaunches.
App type	`LSUIElement` (menu bar only, no Dock icon). URL scheme `stash://` (`clipboard`, `add?text=`, `open`).
Test switches	Env vars: `STASH_LIBRARY` (use another library folder), `STASH_MIGRATE_FROM`, `STASH_LAME`, `STASH_YTDLP`. `./build.sh test` builds "Stash Test" (bundle id `com.yotamkiper.stash.test`): separate settings, separate library, `stashtest://`, no global hot keys; debug hooks only run there.

3. Architecture at a glance

```
 AppDelegate (@main) ── NSStatusItem (menu bar icon, drop target)
      │                ├─ compact StashPanel (borderless, .popUpMenu level) → CompactView
      │                ├─ big StashPanel (titled, resizable, .floating)      → RootView
      │                ├─ Settings panel, Get Info panels, image Preview panel
      │                └─ Carbon global hot keys (⌥Space, quick save ⌥⌘V)
      ▼
 StashPanel.sendEvent ── keys (Delete, arrows, Space, Return, digits, Tab, ^Tab) ── BoardDrag
(mouse)
 DropView (NSView container) ── drops from other apps, ⌘V / ⌘C / ⌘I / ⌘A / ⌘Z / ⌘1–6 / ⌘,
      ▼
 Store (singleton ObservableObject, @MainActor) ── projects, folders, items, selection, UI
state
      │   save() → writeLibrary() (library.json) → scheduleMirror() → syncMirror() →
exportSavedProjects()
      ├─ Undo (snapshot stack)          ├─ Ingest (pasteboard → items, previews, OCR)
      ├─ Mirror (real files on disk)    ├─ Converter (MP3/MP4, downloads)
      ├─ Arrange (free positions)       ├─ ProjectExport (Save to Mac, live)
      └─ ClipHistory (last 5 copies, separate ObservableObject, polls the pasteboard)
```

There is one source of truth in memory, `Store.shared`. Every SwiftUI view reads it through `@EnvironmentObject`. Every change calls `save()`, which rewrites the JSON library and schedules a mirror pass that makes the folder on disk match.

4. Features and behaviour (what must keep working)

4.1 Windows and header

- Compact pop-up hangs under the menu bar icon; it can be dragged by its top bar. **Lock** keeps it exactly there (reopens there); **Unlock** sends it back under the icon. The big window can be locked too (size and place); while it is locked and was the last one open, Stash reopens big.
- Header is two rows: **project tab strip** (Not Selected + pinned/recent projects, as many as fit, ▾ list of all, +), then a **tool row**.
- Tools (each can be hidden in Settings → Toolbar): Search (big window, current tab only, folders and items), 📋 Recent copies, Lock ▾ (pop-up: Lock / Unlock; big window: Draw Frame… / Lock / Unlock — locked = orange 🔒, no resize/move), ❝ minimal/full view (big window), Arrange By ▾ (None, Name, Kind, Date Added, Tags), Icons | List switch, Organize wand, expand/collapse.

- Bottom bar: ⌘1 Note, ⌘2 Folder, ⌘3 Linker, ⌘4 PhotoBook, ⌘5 Notepad, ⌘6 Files, 🗑 Clear (two clicks: empties current tab), ? help.
- Day/night look is automatic by the clock (dark 19:00–07:00).

4.2 The board

- **Icons view**: free placement (Finder "None"): drag anything anywhere and it stays; new things take the first free slot; positions are stored per window and per mode (`compact`, `window`, `window-tiles`). Arrange By lines things up again.
- Pop-up and big-window-minimal show **icon tiles** (icon/thumbnail + name; media links/files get MP3/MP4 chips). Big window with ❝ on shows **full cards** (notes as quote cards with Copy, link previews, file details).
- **List view**: Finder-style rows with stripes.
- Folder icons (design "Dot"): one grey folder shape + a coloured dot for the type; fullness shows sheets peeking out (1–5, 6+) and a count (20+).
- Selection like Finder: click, ⌘/⇧-click, box select, arrows; grey box behind icon + name in a blue pill.

4.3 Keys and mouse

Input	Action
⌥Space (global)	Toggle Stash; right after a screenshot, saves it; with something just copied, opens the matching folder (clipboard banner).
⌥⌘V (global, configurable)	Quick save the clipboard without opening Stash.
Return / ⌘↓ / double-click	Rename / open / open.
Space	Note → open editor; anything else → macOS Quick Look (QLPreviewPanel).
⌘⌫ · ⌫ / Tab / ⌘[	Delete immediately (⌘Z undoes) · go up to the parent folder.
1–9 · ^Tab	Open the numbered folder (numbers follow reading order) · next/previous project tab.
⌘C ⌘V ⌘A ⌘I ⌘Z ⌘⇧Z ⌘F ⌘,	Copy selection · paste into board · select all · Get Info · undo/redo · search · Settings.
Press-hold-release	Rename (notes open editor).
Drag on board	Moves with the pointer instantly (handled in AppKit by `BoardDrag`); over a folder → put inside; out of the window → system drag with fresh visible copies of the real files.

4.4 Content

- **Notes**: rich text editor (font, size, B/I/U, spacing, colour, emoji, lists). Return saves, ⇧Return new line. Saved as .txt or .rtf.
- **Links**: title/preview fetched; YouTube etc. get MP3/MP4. Download progress shows on the link itself; when done, the file **replaces** the link and keeps it as a small badge (open, copy, put back, remove).
- **Files** copied into the library by default (setting), shown with their real Finder icon.
- **Images**: thumbnails, OCR text for search, Quick Look.
- Colour tags (Finder's 7, written to real files), Rename N Items (name + index), Get Info window.
- **Save to Mac…** (project tab menu): keeps a live one-way copy of a project in a folder you choose; only touches what it wrote; removals go to the Trash.
- **Recent copies**: last 5 clipboard items from anywhere (files kept as links only, images as ~50 KB JPEG previews, secrets skipped).

5. Data model (Models.swift)

```
Project  { id, name, createdAt, lastOpened?, pinned? }
Folder   { id, name, style: FolderStyle, projectID?, parentID?, isNamed?, colorTags?, spots?,
createdAt }
Item     { id, kind: link|image|text|file, projectID?, folderID?, createdAt,
           text?, richText (RTF Data)?, url?, title?, imageFile?, sourceURL?, storedPath?,
ocrText?,
           tags: [String], colorTags?, spots? }
BoardSpot { x, y, board: "f:<folder>" | "p:<project or none>" }   // spots: [windowKey:
BoardSpot]
FolderStyle: classic, kraft, pastel, midnight, linker, photos (=PhotoBook), noteHanger
(=Notepad), files
Selectable: .item(UUID) | .folder(UUID)      Place { project?, folder? }      Filter: .inbox
| .project(id) | .all (legacy)
```

Optional fields exist so older libraries still decode. `NotebookPage` / `PageNote` are legacy (the handwriting notebook was removed; code kept in `Unused/Notebook.swift.txt`).

6. Storage on disk

Path	What
`~/Documents/.Stash/`	The library root (hidden by its dot name). Mirrors projects → folders → files.
`…/<Project>/<Folder>/…`	Real files: images, copied files, notes as .txt/.rtf, links as .webloc. Finder colour tags set.
`…/.stash/library.json`	The whole database (pretty-printed, sorted keys, ISO dates). Rewritten completely on every save.
`…/.stash/previews/`	Thumbnails / preview images (`Item.imageFile`).
`…/.stash/staging, trash/<id>`	Mirror moves are two-phase via staging; deleted files wait in trash for 30 days, listed in `trash/deleted.json` (Recently Deleted, with Restore).
`…/.stash/library.prev.json, backups/`	Previous good library version (kept on every save) and hourly backups (newest 24). Used automatically if library.json is damaged.
`…/.stash/exports/`	Save-to-Mac targets (`targets.json`) and per-project manifests.
`…/.stash/clips/`	Recent copies list and small JPEG previews.
`$TMPDIR/StashOut/` , `StashDrag/`	Fresh visible copies handed out on drag-out / Quick Look (cleared at launch); generated link notes (.rtf).
`UserDefaults`	UI settings: layout, sort, pop-up lock and spot, big-window locked frame, quotes, toolbar tools, organizer options, quick-save shortcut, window frames.

7. Source files (what lives where)

File (lines)	Responsibility
AppDelegate.swift (526)	Entry point; status item; creates panels; show/hide/expand/collapse; locking the pop-up in place; settings window; image preview; URL scheme; hot keys; quick save.

Panel.swift (302)	`StashPanel` (event routing, key handling, rename commit on outside click, search focus guard, Quick Look hooks) and `DropView` (drops, ⌘ shortcuts).
Store.swift (995)	State, persistence, queries (`visibleItems/visibleFolders/visibleOrder`), selection, projects, delete, put-in-folder, tags, batch rename, toasts.
Models.swift (174)	Codable data types (section 5).
Undo.swift (110)	Snapshot undo/redo of projects, folders, pages, items (`checkpoint` , `undoGroup`).
Mirror.swift (369)	`syncMirror` : computes target paths, moves/renames/creates files, cleans leftovers, Finder tags, trash; `visibleCopy` for drag-out.
Ingest.swift (371)	Pasteboard/drop → items; link titles & previews; OCR; thumbnails; copy; drag-out providers (`itemProvider` , `folderProvider` , `projectProvider` , link .rtf note).
Converter.swift (588)	MP3/MP4 conversion (AVFoundation → WAV → LAME), yt-dlp downloads with progress, merging a downloaded link into its file, tools settings.
Arrange.swift (206)	Free positions (`moveOnBoard` , `arrange(by:)`), `FreeBoardLayout` (custom SwiftUI Layout), Arrange menu.
BoardDrag.swift (220)	AppKit-level Finder-style dragging: live offset, folder drop, hand-off to NSDraggingSession when leaving the window, press-hold rename.
ProjectExport.swift (136)	Save to Mac: choose location, one-way manifest-based sync on a background queue.
ClipHistory.swift (320)	Last 5 copies: pasteboard polling (0.8 s), file links, JPEG previews, popover UI.
QuickLook.swift (64) · FrameDraw.swift (185)	Space → QLPreviewPanel · draw/lock the big window frame (overlay windows on every screen).
Clipboard.swift (152) · QuickSave.swift (83) · Screenshots.swift (69)	Clipboard banner after ⌥Space · global quick save · catching a fresh screenshot.
RichText.swift (446)	Note editor (NSTextView), toolbar, emoji grid.
Settings.swift (159) · HotKey.swift (41) · Files.swift (46) · Preview.swift (133)	Settings window · Carbon hot keys · file helpers · big image preview window.
Views/BoardView.swift (507)	The board: icons/list, free layout, box select, menus, note composer, empty state, sort enum.
Views/CardView.swift (843)	Every item look: tile, full card, quote card, list row, hover actions, MP3/MP4 chips, progress ring, source-link badge, rename/editor popovers, header toggles.
Views/FolderViews.swift (653)	Folder icon (Dot design), tile/row, fullness icon, folder header (Back/path drop zones), delete prompts, Put in Folder menu.
Views/RootView, CompactView, ProjectBar, AllProjects, QuickBar, Tags, InfoView, PopMenu, ToolbarTools, Appearance, NativeFields, MasonryLayout	Window layouts, tabs and project list, bottom bar, tags & batch rename, Get Info, custom menu buttons, toolbar settings, auto dark mode, `RenameField` (NSTextField), legacy masonry.

8. Key flows

Adding something

Paste / drop / quick save → `Store.ingest(pasteboard, into: Place)` → creates `Item`(s), copies files into the library if enabled, starts async work (link title fetch, thumbnail, OCR) → `save()` → mirror writes the real file.

Any change

`checkpoint("…")` (undo snapshot of all arrays) → mutate arrays → `save()`: encode the whole library to JSON and write it; `scheduleMirror()` coalesces into one `syncMirror()` on the next main-loop turn, then `exportSavedProjects()`.

Dragging out

Board (Icons): `BoardDrag` moves tiles live; leaving the window starts an `NSDraggingSession` whose pasteboard items carry **fresh copies** (`Store.visibleCopy`, APFS clone into `$TMPDIR/StashOut`, hidden flag cleared) plus text/URL. List view and project tabs use SwiftUI `onDrag` with `NSItemProvider` file representations (also fresh copies).

Download

`download(itemID, as:)` runs yt-dlp in a detached task, parses `"% "` lines for progress (`pending` → ring on the link), converts M4A → MP3 when needed, then `finishConversion` replaces the link item in place with the file item (keeps position, tags; `sourceURL` = link).

Delete and undo

`requestDelete` → `performDelete` (folders take their contents) → mirror moves files into `.stash/trash`; ⌘Z restores the snapshot and the mirror moves files back. On the next launch the trash is emptied.

9. Constraints and gotchas (read before changing things)

- **iCloud re-hides library files.** `~/Documents` is synced by iCloud Drive, which sets the "hidden" flag on everything inside the dot-folder `.Stash` within seconds. Never hand library files directly to other apps — always use `visibleCopy`. Moving the library out of `Documents` (e.g. to Application Support with a visible export) would remove this whole class of problems.
- **Recently Deleted.** Deletions are recorded with full metadata and kept 30 days across restarts (`RecentlyDeleted.swift`); undo/redo removals are recorded too.
- **Non-activating panels.** Windows must take keys without activating the app; SwiftUI swallows Space/Delete, so keys are handled in `StashPanel.sendEvent`. Mouse dragging on the board is intercepted there too (`BoardDrag`), so SwiftUI `onDrag` only works in List view.
- **SwiftUI focus quirks.** Rename uses an AppKit `NSTextField` (`RenameField`); search can only take focus on click or ⌘F (guard in `makeFirstResponder`).
- **SwiftUI menus** with `.borderlessButton` draw only an icon, so header menu buttons are custom (`MenuIconButton` + `NSMenu`).
- **Mirror is authoritative for files.** Item file paths (`storedPath`) change when things move or are renamed; always resolve through `contentURL` / `dragFile`.
- **Code signing** fails on iCloud-synced folders (extended attributes) — build.sh signs in a temp folder.
- **Saving** is asynchronous on a serial queue (`Persistence.swift`): numbered snapshots, newest wins, read-back check, atomic replace with `library.prev.json`, hourly backups, recovery at launch. Flushed in `applicationWillTerminate`.

10. Where it can be made more efficient

Ordered by expected impact. High = noticeable with large libraries; Medium ; Low .

#	Area	Today → suggestion
1	Partly done Whole-library save on every change	Saves now encode and write on a background queue with coalescing, read-back check, atomic replace and backups (`Persistence.swift`); main-thread cost <1 ms. Still a full rewrite (~40–60 ms in the background for 2,000 items). → Debounce saves (e.g. 300 ms), encode off the main thread, drop pretty-printing, or move to SQLite/GRDB or one file per project.
2	Partly done Full mirror pass after every save	Now fingerprinted (Oct 2026): `mirrorPrints` skip disk checks, note-content reads and tag reads for unchanged items; full audit at launch and every 5 min. 2,000 items: 207–280 ms → ~35 ms. Next: move the remaining CPU work (paths, unique names) off the main thread.
3	High One giant ObservableObject	Every `@Published` change (selection, toast, drag offset, search text) re-evaluates every view that reads `Store`. → Split into smaller observable models (library data, UI/selection, drag state) or adopt the Observation framework (`@Observable`, macOS 14) so views only update when what they read changes.
4	Done Drag redraws	Fixed (Oct 2026): drag state lives in `DragSelection` / `DragOffset`, not Store; a snapshot "ghost" NSImageView follows the pointer. 2,000-item board: ~450 ms/step → 16.7 ms/frame, 0 card redraws per frame.

5	Medium Repeated computed queries	`visibleItems` / `visibleFolders` / `visibleOrder`, `count(in:)`, `tabProjects` (called per `ViewThatFits` candidate) filter whole arrays every render. → Cache derived indexes (items by folder/project, counts) and invalidate on change; use dictionaries keyed by ID instead of `first(where:)`.
6	Medium Undo snapshots copy everything	`checkpoint` stores full copies of all arrays (cheap copy-on-write until mutated, but each mutation then duplicates). → Store diffs (inverse operations) or cap memory by size.
7	Partly done Images and icons in render paths	`displayImage`, `NSWorkspace.icon(forFile:)`, `previewImages`, file sizes in cards are fetched during view bodies. → Cache thumbnails at display size (NSCache keyed by item + size), generate with QuickLookThumbnailing off-main, prefetch.
8	Medium FreeBoardLayout free-slot search	Places each unpositioned item by scanning rows in 8 pt steps against all placed rects ($O(n^2)$). → Use a grid occupancy map or only scan near the end of the content; cache layout results in the Layout cache.
9	Medium Save-to-Mac sync walks everything	Each change walks the whole project folder and compares sizes/dates. → Reuse the mirror's change set (item 2) to copy only what changed; FSEvents for external edits.
10	Low Clipboard polling	Timer every 0.8 s reading `changeCount` (cheap). → Fine; could pause while the Mac is idle/asleep.
11	Low Drag-out copies	`visibleCopy` clones files at hand-off (instant on APFS, but folder clones walk trees). → Create lazily with `NSFilePromiseProvider`, only when the receiver accepts the drop.
12	Low Big files in Views	`CardView.swift` (843 lines) mixes many looks. → Split into TileView, CardView, QuoteCard, RowView for faster compiles and clearer diffs.

Disk work done (Oct 2026): previews capped at 600 px JPEG/PNG (`Storage.swift`, one-time shrink of old ones), thumbnail NSCache limits, yt-dlp run from the bundle (copy only on update), lzfse-compressed hourly backups, temporary folders cleared at launch (separate ones for Stash Test), and a Settings → Storage report with safe clean-up actions.

Biggest single win: items 1–3 together — debounced background persistence, an incremental mirror, and finer-grained observation — keep the UI smooth as libraries grow into thousands of items.

---

11. Suggested questions to ask an AI with this brief

- "Design an incremental version of `syncMirror` that only processes changed IDs. Show the data structures and where to hook it into `save()`."
- "Migrate `Store` from `ObservableObject` to the Observation framework without changing behaviour. What breaks?"
- "Should the library move out of iCloud Documents? Propose a safe migration that keeps a visible folder for the user."
- "Replace JSON persistence with SQLite (GRDB) — schema, migration from library.json, and expected speed-up."
- "Build a thumbnail cache using QuickLookThumbnailing and NSCache for cards and tiles."